Switches used in a computer make use of logic gates (see Chapter 10) and are used to store and process data.

1.1.2 Binary, denary and hexadecimal systems

The binary system

We are all familiar with the denary number system which counts in multiples of 10. This gives us the well-known headings of units, 10s, 100s, 1000s, and so on:

(10^4)	(10^3)	(10^2)	(10^1)	(10^0)
10 000	**1000**	**100**	**10**	**1**
2	5	1	7	7

Denary uses ten separate digits, 0-9, to represent all values. Denary is known as a base 10 number system.

The **binary number system** is a base 2 number system. It is based on the number 2. Thus, only the two 'values' 0 and 1 can be used in this system to represent all values. Using the same method as denary, this gives the headings 2^0, 2^1, 2^2, 2^3, and so on. The typical headings for a binary number with eight digits would be:

(2^7)	(2^6)	(2^5)	(2^4)	(2^3)	(2^2)	(2^1)	(2^0)
128	**64**	**32**	**16**	**8**	**4**	**2**	**1**
1	1	1	0	1	1	1	0

A typical binary number would be: 11101110.

Converting from binary to denary

The conversion from binary to denary is a relatively straightforward process. Each time a 1-value appears in a binary number column, the column value (heading) is added to a total. This is best shown by three examples which use 8-bit, 12-bit and 16-bit binary numbers:

Example 1

Convert the binary number, 11101110, into a denary number.

128	64	32	16	8	4	2	1
1	1	1	0	1	1	1	0

The equivalent denary number is 128 + 64 + 32 + 8 + 4 + 2 = 238

Example 2

Convert the following binary number, 011110001011, into a denary number.

2048	1024	512	256	128	64	32	16	8	4	2	1
0	1	1	1	1	0	0	0	1	0	1	1

The equivalent denary number is 1024 + 512 + 256 + 128 + 8 + 2 + 1 = 1931

Example 3

Convert the following binary number, 0011000111100110, into a denary number.

32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
0	0	1	1	0	0	0	1	1	1	1	0	0	1	1	0

As with the two examples above, to convert this number to denary, each time a 1 appears in a column the column value is added to the total:

8192 + 4096 + 256 + 128 + 64 + 32 + 4 + 2 = 12 774

The same method can be used for a binary number of any size.

Activity 1.1

Convert the following binary numbers into denary:

a 0 0 1 1 0 0 1 1
b 0 1 1 1 1 1 1 1
c 1 0 0 1 1 0 0 1
d 0 1 1 1 0 1 0 0
e 1 1 1 1 1 1 1 1
f 0 0 0 0 1 1 1 1
g 1 0 0 0 1 1 1 1
h 1 0 1 1 0 0 1 1
i 0 1 1 1 0 0 0 0
j 1 1 1 0 1 1 1 0
k 0001 1110 0111
l 0101 0101 0100
m 1111 0000 1111
n 0111 1100 1000
o 0111 1111 1111
p 0111 1100 1111 0000
q 0011 1111 0000 1101
r 1100 0011 0011 1111
s 1000 1000 1000 1000
t 0111 1111 1111 1111

Converting from denary to binary

The conversion from denary numbers to binary numbers can be done in two different ways. The first method involves successive subtraction of powers of 2 (that is, 128, 64, 32, 16, and so on); whilst the second method involves successive division by 2 until the value "0" is reached. This is best shown by two examples:

Example 1

Consider the conversion of the denary number, 142, into binary:

Method 1

The denary number 142 is made up of 128 + 8 + 4 + 2 (that is, 142 – 128 = 14; 14 – 8 = 6; 6 – 4 = 2; 2 – 2 = 0; in each stage, subtract the largest possible power of 2 and keep doing this until the value 0 is reached. This will give us the following 8-bit binary number:

128	64	32	16	8	4	2	1
1	0	0	0	1	1	1	0

Method 2

This method involves successive division by 2. Start with the denary number, 142, and divide it by 2. Write the result of the division including the remainder (even if it is 0) under the 142 (that is, 142 ÷ 2 = 71 remainder 0); then divide again by 2 (that is, 71 ÷ 2 = 35 remainder 1) and keep dividing until the result is zero. Finally write down all the remainders in reverse order:

2	142				read the remainders from bottom to top to get the binary number: **1 0 0 0 1 1 1 0**
2	71	remainder:	0		
2	35	remainder:	1		
2	17	remainder:	1		
2	8	remainder:	1		
2	4	remainder:	0		
2	2	remainder:	0		
2	1	remainder:	0		
	0	remainder:	1		

▲ **Figure 1.1**

We end up with an 8-bit binary number which is the same as that found by Method 1.

Example 2

Consider the conversion of the denary number, 59, into binary:

Method 1

The denary number 59 is made up of 32 + 16 + 8 + 2 + 1 (that is, 59 – 32 = 27; 27 – 16 = 11; 11 – 8 = 3; 3 – 2 = 1; 1 – 1 = 0; in each stage, subtract the largest possible power of 2 and keep doing this until the value 0 is reached. This will give us the following 8-bit binary number:

128	64	32	16	8	4	2	1
0	0	1	1	1	0	1	1

Method 2

This method involves successive division by 2. Start with the denary number, 59, and divide it by 2. Write the result of the division including the remainder (even if it is 0) under the 59 (that is, 59 ÷ 2 = 29 remainder 1); then divide again by 2 (that is, 29 ÷ 2 = 14 remainder 1) and keep dividing until the result is zero. Finally write down all the remainders in reverse order:

2	59				write the remainders from bottom to top to get the binary number: **1 1 1 0 1 1**
2	29	remainder:	1		
2	14	remainder:	1		
2	7	remainder:	0		
2	3	remainder:	1		
2	1	remainder:	1		
	0	remainder:	1		

▲ **Figure 1.1b**

If we want to show this as an 8-bit binary number (as shown in Method 1), we now simply add two 0's from the left-hand side to give the result: 0 0 1 1 1 0 1 1. The two results from both methods clearly agree.

Both the above examples use 8-bit binary numbers. This third example shows how the method can still be used for any size of binary number; in this case a 16-bit binary number.

Example 3

Consider the conversion of the denary number, 35 000, into a 16-bit binary number:

Method 1

The denary number 35 000 is made up of 32 768 + 2048 + 128 + 32 + 16 + 8 (that is, 35 000 – 32 768 = 2232; 2232 – 2048 = 184; 184 – 128 = 56; 56 – 32 = 24; 24 – 16 = 8; 8 – 8 = 0; in each stage, subtract the largest possible power of 2 and keep doing this until the value 0 is reached. This will give us the following 16-bit binary number:

32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
1	0	0	0	1	0	0	0	1	0	1	1	1	0	0	0

Method 2

This method involves successive division by 2. Start with the denary number, 35000, and divide it by 2. Write the result of the division including the remainder (even if it is 0) under the 35 000 (that is, 35 000 ÷ 2 = 17 500 remainder 0); then divide again by 2 (that is, 17 500 ÷ 2 = 8750 remainder 0) and keep dividing until the result is zero. Finally write down all the remainders in reverse order:

2	35 000		
2	17 500	remainder:	0
2	8750	remainder:	0
2	4375	remainder:	0
2	2187	remainder:	1
2	1093	remainder:	1
2	546	remainder:	1
2	273	remainder:	0
2	136	remainder:	1
2	68	remainder	0
2	34	remainder	0
2	17	remainder	0
2	8	remainder	1
2	4	remainder	0
2	2	remainder	0
2	1	remainder	0
	0	remainder	1



read the remainder from bottom to top to get the binary number:

1 0 0 0 1 0 0 0 1 0
1 1 1 0 0 0

▲ **Figure 1.1c**

Activity 1.2

Convert the following denary numbers into binary (using both methods):

a 41	**d** 100	**g** 144	**j** 255	**m** 4095
b 67	**e** 111	**h** 189	**k** 33000	**n** 16400
c 86	**f** 127	**i** 200	**l** 888	**o** 62307

The hexadecimal system

The **hexadecimal number system** is very closely related to the binary system. Hexadecimal (sometimes referred to as simply 'hex') is a base 16 system and therefore needs to use 16 different 'digits' to represent each value.

Because it is a system based on 16 different digits, the numbers 0 to 9 and the letters A to F are used to represent each hexadecimal (hex) digit. A in hex = 10 in denary, B = 11, C = 12, D = 13, E = 14 and F = 15.

Using the same method as for denary and binary, this gives the headings 16^0, 16^1, 16^2, 16^3, and so on. The typical headings for a hexadecimal number with five digits would be:

(16^4)	(16^3)	(16^2)	(16^1)	(16^0)
65 536	**4096**	**256**	**16**	**1**
2	1	F	3	A

A typical example of hex is 2 1 F 3 A.

Since $16 = 2^4$ this means that FOUR binary digits are equivalent to each hexadecimal digit. The following table summarises the link between binary, hexadecimal and denary:

▼ **Table 1.1**

Binary value	Hexadecimal value	Denary value
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Converting from binary to hexadecimal and from hexadecimal to binary

Converting from binary to hexadecimal is a fairly easy process. Starting from the right and moving left, split the binary number into groups of 4 bits. If the last group has less than 4 bits, then simply fill in with 0s from the left. Take each group of 4 bits and convert it into the equivalent hexadecimal digit using Table 1.1. Look at the following two examples to see how this works.

Example 1

1 0 1 1 1 1 1 0 0 0 0 1

First split this up into groups of 4 bits:

1 0 1 1 1 1 1 0 0 0 0 1

Then, using Table 1.1, find the equivalent hexadecimal digits:

B E 1

Example 2

1 0 0 0 0 1 1 1 1 1 1 1 0 1

First split this up into groups of 4 bits:

1 0 0 0 0 1 1 1 1 1 1 0 1

The left group only contains 2 bits, so add in two 0s:

0 0 1 0 0 0 0 1 1 1 1 1 1 0 1

Now use Table 1.1 to find the equivalent hexadecimal digits:

2 1 F D

Activity 1.3

Convert the following binary numbers into hexadecimal:

a 1 1 0 0 0 0 1 1
b 1 1 1 1 0 1 1 1
c 1 0 0 1 1 1 1 1 1 1
d 1 0 0 1 1 1 0 1 1 1 0
e 0 0 0 1 1 1 1 0 0 0 0 1
f 1 0 0 0 1 0 0 1 1 1 1 0
g 0 0 1 0 0 1 1 1 1 1 1 1 0
h 0 1 1 1 0 1 0 0 1 1 1 0 0
i 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1
j 0 0 1 1 0 0 1 1 1 1 0 1 0 1 1 1 0

Converting from hexadecimal to binary is also very straightforward. Using the data in Table 1.1, simply take each hexadecimal digit and write down the 4-bit code which corresponds to the digit.

Example 3

4	5	A

Using Table 1.1, find the 4 bit code for each digit:

0 1 0 0	0 1 0 1	1 0 1 0

Put the groups together to form the binary number:

0 1 0 0 0 1 0 1 1 0 1 0

Example 4

B	F	0	8

Again just use Table 1.1:

1 0 1 1	1 1 1 1	0 0 0 0	1 0 0 0

Then put all the digits together:

1 0 1 1 1 1 1 1 0 0 0 0 1 0 0 0

Activity 1.4

Convert the following hexadecimal numbers into binary:

a 6 C
b 5 9
c A A
d A 0 0
e 4 0 E
f B A 6
g 9 C C
h 4 0 A A
i D A 4 7
j 1 A B 0

Converting from hexadecimal to denary and from denary to hexadecimal

To ***convert hexadecimal numbers into denary*** involves the value headings of each hexadecimal digit; that is, 4096, 256, 16 and 1.

Take each of the hexadecimal digits and multiply it by the heading values. Add all the resultant totals together to give the denary number. Remember that the hex digits A → F need to be first converted to the values 10 → 15 before carrying out the multiplication. This is best shown by two examples:

Example 1

Convert the hexadecimal number, 4 5 A, into denary.

First of all we have to multiply each hex digit by its heading value:

256	16	1	
4	5	A	
(4 × 256 = 1024)	(5 × 16 = 80)	(10 × 1 = 10)	(NOTE: A = 10)

Then we have to add the three totals together (1024 + 80 + 10) to give the denary number:

1 1 1 4

Example 2

Convert the hexadecimal number, C 8 F, into denary.

First of all we have to multiply each hex digit by its heading value:

256	16	1	
C	8	F	
(12 × 256 = 3072)	(8 × 16 = 128)	(15 × 1 = 15)	(NOTE: C = 12, F = 15)

Then we have to add the three totals together (3072 + 128 + 15) to give the denary number:

3 2 1 5

Activity 1.5

Convert the following hexadecimal numbers into denary:

a 6 B
b 9 C
c 4 A
d F F
e 1 F F
f A 0 1
g B B 4
h C A 8
i 1 2 A E
j A D 8 9

To ***convert from denary to hexadecimal*** involves successive division by 16 until the value "0" is reached. This is best shown by two examples:

Example 1

Convert the denary number, 2004, into hexadecimal.

This method involves successive division by 16 until the value 0 is reached. We start by dividing the number 2004 by 16. The result of the division including the remainder (even if it is 0) is written under 2004 and then further divisions by 16 are carried out (that is, 2004 ÷ 16 = 125 remainder 4; 125 ÷ 16 = 7 remainder 13; 7 ÷ 16 = 0 remainder 7). The hexadecimal number is obtained from the remainders written in reverse order:

16	2004				write the remainders from bottom to top to get the hexadecimal number:
16	125	remainder:	4		
16	7	remainder:	13		7 D 4 (D=13)
	0	remainder:	7		

▲ **Figure 1.2a**

Example 2

Convert the denary number, 8463, into hexadecimal.

We start by dividing the number 8463 by 16. The result of the division including the remainder (even if it is 0) is written under 8463 and then further divisions by 16 are carried out (that is, 8463 ÷ 16 = 528 remainder 15; 528 ÷ 16 = 33 remainder 0; 33 ÷ 16 = 2 remainder 1; 2 ÷ 16 = 0 remainder 2). The hexadecimal number is obtained from the remainders written in reverse order:

16	8463				read the remainder from bottom to top to get the hexadecimal number:
16	528	remainder:	15		
16	33	remainder:	0		2 1 0 F (F=15)
16	2	remainder:	1		
	0	remainder:	2		

▲ **Figure 1.2b**

Activity 1.6

Convert the following denary numbers into hexadecimal:

a	98	f	1000
b	227	g	2634
c	490	h	3743
d	511	i	4007
e	826	j	5000

1.1.3 Use of the hexadecimal system

As we have seen, a computer can only work with binary data. Whilst computer scientists can work with binary, they find hexadecimal to be more convenient to use. This is because one hex digit represents four binary digits. A complex binary number, such as 1101001010101111 can be written in hex as D2AF. The hex number is far easier for humans to remember, copy and work with. This section reviews four uses of the hexadecimal system:

- error codes
- MAC addresses
- IPv6 addresses
- HTML colour codes

The information in this section gives the reader sufficient grounding in each topic at this level. Further material can be found by searching the internet, but be careful that you don't go off at a tangent.

Error codes

Error codes are often shown as hexadecimal values. These numbers refer to the memory location of the error and are usually automatically generated by the computer. The programmer needs to know how to interpret the hexadecimal error codes. Examples of error codes from a Windows system are shown below:

```
e HexErrorCode ErrorDescription
- ------------ ----------------
0 0x0          Success
1 0x1          Incorrect function.
2 0x2          The system cannot find the file specified.
3 0x3          The system cannot find the path specified.
4 0x4          The system cannot open the file.
5 0x5          Access is denied.
6 0x6          The handle is invalid.
7 0x7          The storage control blocks were destroyed.
8 0x8          Not enough storage is available to process this command.
9 0x9          The storage control block address is invalid.
0 0xa          Unit test error string
1 0xb          An attempt was made to load a program with an incorrect format.
2 0xc          The access code is invalid.
3 0xd          The data is invalid.
4 0xe          Not enough storage is available to complete this operation.
5 0xf          The system cannot find the drive specified.
6 0x10         The directory cannot be removed.
7 0x11         The system cannot move the file to a different disk drive.
8 0x12         There are no more files.
9 0x13         The media is write protected.
0 0x14         The system cannot find the device specified.
1 0x15         The device is not ready.
2 0x16         The device does not recognize the command.
3 0x17         Data error (cyclic redundancy check).
4 0x18         The program issued a command but the command length is incorrect.
5 0x19         The drive cannot locate a specific area or track on the disk.
6 0x1a         The specified disk or diskette cannot be accessed.
7 0x1b         The drive cannot find the sector requested.
8 0x1c         The printer is out of paper.
9 0x1d         The system cannot write to the specified device.
0 0x1e         The system cannot read from the specified device.
1 0x1f         A device attached to the system is not functioning.
2 0x20         The process cannot access the file because it is being used by another process.
3 0x21         The process cannot access the file because another process has locked a portion of the file.
4 0x22         The wrong diskette is in the drive....
6 0x24         Too many files opened for sharing.
8 0x26         Reached the end of the file.
9 0x27         The disk is full.
0 0x32         The request is not supported.
```

▲ **Figure 1.3** Example of error codes

Find out more

Another method used to trace errors during program development is to use memory dumps, where the memory contents are printed out either on screen or using a printer. Find examples of memory dumps and find out why these are a very useful tool for program developers.